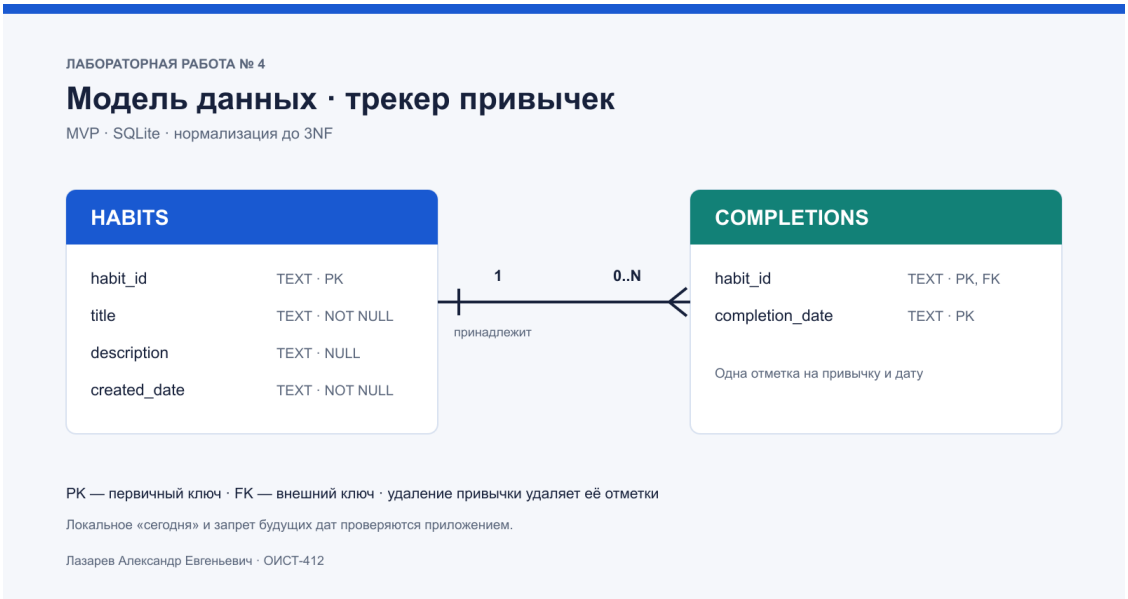
Проектирование и нормализация базы данных



Лабораторная работа № 4. Проект «Трекер полезных привычек».

Выполнил: Лазарев Александр Евгеньевич, ОИСТ-412.

1. Предметная область и границы

MVP позволяет создать, изменить и удалить ежедневную привычку, отметить или снять её выполнение за доступный день, открыть историю, серии и месячный прогресс. По командной доске в MVP есть две сущности: привычка (habits) и отметка (completions). Привычка может иметь ноль или много отметок; каждая отметка принадлежит ровно одной привычке. Запись отметки означает «выполнено»; отсутствие записи означает «не отмечено» для доступной даты.

Рабочие правила: название после удаления крайних пробелов — 1–100 символов; описание необязательно и до 500 символов; дата создания — локальная календарная дата пользователя; на одну привычку и день — не более одной отметки. Даты раньше создания и будущие даты не должны приниматься. Удаление привычки удаляет её историю. На доске часовой пояс, удаление истории и включение текущего дня в месячный процент помечены как решения для согласования с Product Owner.

Аккаунт, синхронизация, категории и расписание не включены в MVP: вход и категории находятся в расширенном бэклоге, а модель идентификации ещё не согласована. Поэтому в основной схеме нет вымышленных пользователей или связи многие-ко-многим. Если позднее появится требование нескольких категорий у одной привычки, нужна будет таблица связей habit_categories(habit_id, category_id) с составным первичным ключом; сейчас это только пример возможного расширения.

2. Исходная структура и аномалии

Для анализа представим одну таблицу:

Habit_Log(habit_id, title, description, created_date, completion_date).

Если даты записаны списком в одной ячейке, их сложно проверять и обновлять независимо. Если хранить по одной дате в строке, название и описание повторяются для каждой отметки. Их изменение потребует обновить все строки; ошибка оставит противоречивые значения. Удаление последней отметки может удалить и данные о привычке. Привычку без отметок тоже неудобно представить без специальной пустой строки.

3. Первая нормальная форма — 1NF

Значения делаются атомарными: одна строка содержит одну календарную дату, а не список дат. Для такой временной таблицы естественный составной ключ — (habit_id, completion_date). Повторение данных привычки при этом остаётся; 1NF само по себе не устраняет избыточность.

4. Вторая нормальная форма — 2NF

В таблице с ключом (habit_id, completion_date) название, описание и дата создания зависят только от части ключа — habit_id, а не от всей пары. Это частичная зависимость. Поэтому выносим их в habits(habit_id, title, description, created_date), а в completions(habit_id, completion_date) оставляем только пару, обозначающую выполнение за день. Ключ привычки простой; у отметки составной ключ, а неключевых полей в ней нет. Частичных зависимостей не остаётся.

5. Третья нормальная форма — 3NF

Функциональные зависимости: habit_id → title, description, created_date; (habit_id, completion_date) → факт наличия отметки. В предложенных таблицах нет неключевого атрибута, который определял бы другой неключевой атрибут. Текущая и лучшая серии, количество и процент выполнения вычисляются по отметкам и выбранной дате, а не дублируются в habits: иначе после правки истории они могли бы устареть. При указанных бизнес-правилах обе таблицы находятся в 3NF.

6. Ключи, связи и ограничения

– habits.habit_id — первичный ключ; строковый UUID задаётся приложением.

– completions(habit_id, completion_date) — составной первичный ключ, который запрещает дубликат за день.

– completions.habit_id — внешний ключ на habits.habit_id, ON DELETE CASCADE; его действие в SQLite включается через PRAGMA foreign_keys = ON отдельно для каждого подключения.

– Проверки CHECK ограничивают длину названия и описания, а также формат и корректность дат. Триггеры запрещают отметку раньше создания и изменение даты создания привычки.

– Приложение должно определить локальное «сегодня», отклонить будущую дату, формировать UUID и нормализовать пробелы Unicode. SQL не может самостоятельно корректно определить пользовательскую локальную дату без контекста часового пояса. Встроенный trim проверяет обычные пробелы, табуляцию и переносы строк; полная обработка Unicode относится к приложению.

– Индекс первичного ключа (habit_id, completion_date) поддерживает историю одной привычки; дополнительные индексы по дате создания и дате отметки помогают соответствующим выборкам.

– Повтор постановки отметки реализуется в приложении запросом INSERT ... ON CONFLICT(habit_id, completion_date) DO NOTHING; повторное снятие через DELETE не создаёт ошибку при отсутствии строки.

7. Итоговая схема и проверка

Схема: ER-диаграмма. Данные создаются скриптом schema.sql, демонстрационные записи — отдельным seed_data.sql. Тесты создают базу в памяти, проверяют внешние ключи, атомарность дат, ограничения длины, уникальность, диапазон от даты создания, каскадное удаление и пример месячного процента 12/20 = 60%. Проверка будущей даты остаётся обязанностью приложения и не представлена как гарантия SQLite.

Проектирование здесь представляет автономный учебный артефакт: существующего серверного приложения и подключения этой схемы к нему не заявлено.